

一、C++与 C 编程规范

在自动化学院的各专业实践课程中，要求学生遵循编程规范具有深远意义。一方面，编程规范是代码世界的**默认规则**，在学习阶段养成规范编程的习惯，代码具备统一的**语法结构、命名规则与注释风格**，有助于日后在**阅读 AI 生成源码、协作团队代码**的过程中，迅速理解**代码逻辑与功能意图**，降低理解成本；另一方面，规范的编程习惯能**有效减少**代码中的**潜在错误**，提升代码的**可读性与可维护性**，为学生未来进入实际工作场景、参与大型项目开发打下坚实基础。此外，随着技术迭代加速，自动化行业对开发者的代码质量与协作效率要求日益严苛，学生在校掌握编程规范，可更好地与行业标准接轨，增强自身竞争力，实现从校园学习到职场实践的无缝过渡。

1. 命名规范

1.1 代码中的命名**严禁**以下**划线_**或**美元符号\$**开始和结束。

反例：_name/_name/\$Object/name_/name\$/Object\$

正例：m_speed/adcValue/system_tick（C++成员变量、局部变量和 C 变量）

1.2 代码中的命名**严禁**使用**拼音与英文混合**的方式，更不允许直接使用中文的方式（正确的英文拼写习惯让读者易于理解，避免歧义）。

反例：DaZhePromotion[打折]/getPingfenByName()[评分]/int 分数=30

正例：taobao/changsha/youku 等国际通用的名称，可视同英文

1.3 **常量**命名**全部大写**，单词间用下划线隔开，力求语义表达完整清楚，不要嫌名字长。

正例：MAX_CHAR_COUNT/ADC_SAMPLE_RATE

反例：MAX_COUNT/SMP_RATE

1.4 杜绝完全不规范的**缩写**，避免望文不知义。

反例：condition 缩写成 condi，temporary 缩写成 temp，严重降低了代码可读性

1.5 C++与 C **类命名**存在差异：

（1）C++**类名**使用**大驼峰命名法**(UpperCamelCase)，但专业领域关键词命名例外，如 PID/PI/PWM 等，也不允许两个关键词连续出现（若关键词本身就存在大小写、**半全称**格式，如 I2C/i2c、ESP8266/esp8266、TCP/Tcp、UDP/Udp，则按需书写）。

正例：SensorData/PIDController/TcpUdpSocket

反例：sensorData/pidController/TCPUDPSocket

（2）C 语言**结构体名**多使用**蛇形命名法**(全小写+下划线)，或**大驼峰命名法**（需配合 typedef struct 使用）。

正例：struct gpio_pin/typedef struct SensorData {...} SensorData;

1.6 C++**抽象类**命名使用 Abstract 或 Base 开头，**异常类**命名以 Exception 结尾。

正例: AbstractController / IOException

1.7 C++与 C 的函数命名区别

(1) C++类中成员函数使用小驼峰命名法(lowerCamelCase), 类外部的全局函数(较少使用)推荐蛇形命名法(全小写+下划线)。

正例: class Sensor::getTemperature() / system_init()

(2) C 中函数统一使用蛇形命名法(全小写+下划线)。

正例: gpio_init() / adc_read_value()

1.8 C++与 C 的变量命名区别

(1) C++中:

成员变量: 前缀 m_+小驼峰, 以区分成员变量(member)和局部变量。

正例: int m_currentSpeed; / float m_temperature;

局部变量和参数: 仅使用小驼峰, 不使用前缀 m_。

正例: void setPosition(int xCoord, int yCoord);

(2) C 中所有变量和函数统一使用蛇形命名法(全小写+下划线)。

正例: int adc_value;/void process_data(uint8_t* buffer);

1.9 C++与 C 的源文件命名

C++源文件使用蛇形命名法 (.cpp)

正例: gpio_driver.cpp / adc_reader.cpp

C++头文件使用蛇形命名法 (.h 或.hpp)

正例: i2c_master.h / pid_controller.hpp

1.10 类中函数命名规约:

- ①获取单个对象的函数用 get 做前缀;
- ②获取多个对象的函数用 list 做前缀;
- ③获取统计值的函数用 count 做前缀;
- ④插入数据库中值的函数用 save 或 insert 做前缀;
- ⑤删除数据库中值的函数用 remove 或 delete 做前缀;
- ⑥修改数据库中值的函数用 update 做前缀。

2. 代码结构规范

2.1 空格、大括号格式规范

二元运算符(如=、+、-、*、/)两侧各有一个空格: a = b + c;

逗号后一个空格: func(arg1, arg2);

函数名与参数列表之间的括号无空格: void func(int a);

大括号典型的两种格式: **Attach 风格**是左大括号不要换行(行尾大括号), **Allman 风格**是左大括号要换行(行首大括号), 前者代码更紧凑, 节省行数, 后者层次结构清晰, 一

目了然。

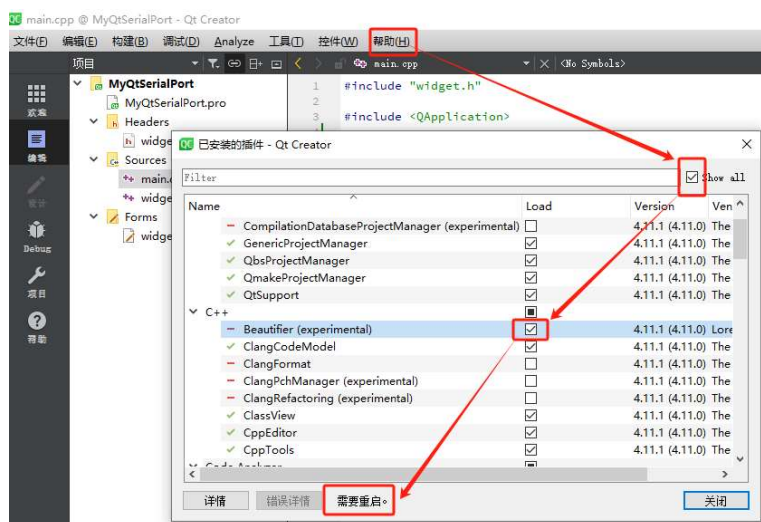
规定：在一个项目内使用一种统一的风格，不要混用！

Attach 风格（推荐，书写会更快）	Allman 风格（也可）
<pre>if (condition) { ...//代码块内部要缩进 } else { ...//代码块内部要缩进 }</pre>	<pre>if (condition) { ...//代码块内部要缩进 } else { ...//代码块内部要缩进 }</pre>

2.2 代码块内部缩进和括号前后空格的格式规范

Qt 中同时按下 **Ctrl+i** 可完成代码块内部缩进的格式化。但若要实现括号前后空格的格式化，还需要安装**格式化插件**。

帮助 > 关于插件 > C ++ > Beautifier 启用该插件。



选择工具 > 选项 > Beautifier，设置中勾选“Enable auto format on file save”复选框。。。待落实！

2.3 源文件与头文件

C++和 C 的**声明**与**实现**分离（.h/.cpp、.h/.c），头文件声明**类**、**函数**、**常量**、**类型定义**等接口，源文件**实现**头文件中声明的功能。

头文件使用**#pragma once**或**传统宏保护**防止头文件重复包含。

//方式 1: #pragma once（现代编译器支持）

```
#pragma once
class Sensor { ... };
```

//方式 2: 传统宏保护

```
#ifndef SENSOR_H
#define SENSOR_H
class Sensor { ... };
#endif
```

2.4 目录结构划分

将工程目录按功能模块分门别类划分，避免文件过多导致结构混乱。

```
project/
├── inc/           # 头文件目录
│   ├── core/     # 核心功能
│   ├── drivers/  # 驱动层
│   └── utils/     # 工具类
└── src/          # 源文件目录
    ├── core/
    ├── drivers/
    └── utils/
```

2.5 C++代码块结构支持类成员访问控制

按 public → protected → private 顺序排列成员，成员变量放在类尾部，与函数分离，增强可读性

```
class MotorDriver {
public:    // 公共函数
    void initialize();
    void setSpeed(uint8_t speed);
protected: // 受保护成员函数
    virtual void updatePWM();
private:   // 私有成员变量
    uint16_t m_pwmDutyCycle;
};
```

//命名空间特色

```
namespace Embedded {
    namespace MotorControl {
        class Driver { ... };
    }
}
// 使用
Embedded::MotorControl::Driver motor;
```

2.6 代码注释

(1) 文件首行注释

新增或修改文件，要在首行添加文件版权、功能概述、作者信息、日期等注释说明。

```
/**
 * @file MotorController.h
 * @brief 电机控制模块
 * @author 张三
 * @date 2025-06-15
 */
```

(2) 函数注释

新增或修改函数，要在函数前添加功能概述、函数参数、返回值、异常等注释说明。

Qt 自带 Doxygen 格式的注释，只需输入/**并回车，即可自动生成函数注释。

```
/**
 * @brief 计算 PID 输出值
```

```

* @param[in] setpoint 目标值
* @param[in] current 当前值
* @return 计算后的输出值
*/
float calculatePID(float setpoint, float current);

```

3. 编程实践规范

3.1 动态内存分配

C++使用 new/delete 或智能指针（如 std::unique_ptr），前者需手动管理内存。

```

Sensor* sensor = new TemperatureSensor();
delete(sensor);

std::unique_ptr<Sensor> sensor = std::make_unique<TemperatureSensor>();

```

C 使用 malloc/free，需手动管理内存。

```

Sensor* sensor = (Sensor*)malloc(sizeof(TemperatureSensor));
free(sensor);

```

3.2 静态与全局变量

C++类的静态成员变量需在源文件中初始化。

```

//MyClass.h
class MyClass {
    static int counter;
};

//MyClass.cpp
int MyClass::counter = 0;

```

C 中全局变量直接在头文件(.h)或源文件(.c)中定义。

3.3 C++使用 virtual 关键字实现多态

```

class Shape {
public:
    virtual float Area() const = 0; // 父类纯虚函数
};

class Circle: public Shape { // 子类继承父类
public:
    float Area() const override { // 重载 Area 函数
        return 3.14f*m_radius*m_radius;
    }
};

```

3.4 C++构造函数与析构函数

```

class ResourceManager {
public:
    ResourceManager() { /* 初始化资源 */ }
    ~ResourceManager() { /* 释放资源 */ }
};

```

3.5 C++使用异常机制来处理错误。

```

void ReadSensor() {

```

```

... //生成一个 error
if (error) {
    throw std::runtime_error("Sensor read failed"); //内层抛出异常
}
}
try {
    ReadSensor();
} catch (const std::exception& e) { //外层捕获异常
    // 处理异常
}

```

而微控制器 C 中慎用抛异常机制，使用错误码或回调函数处理错误

```

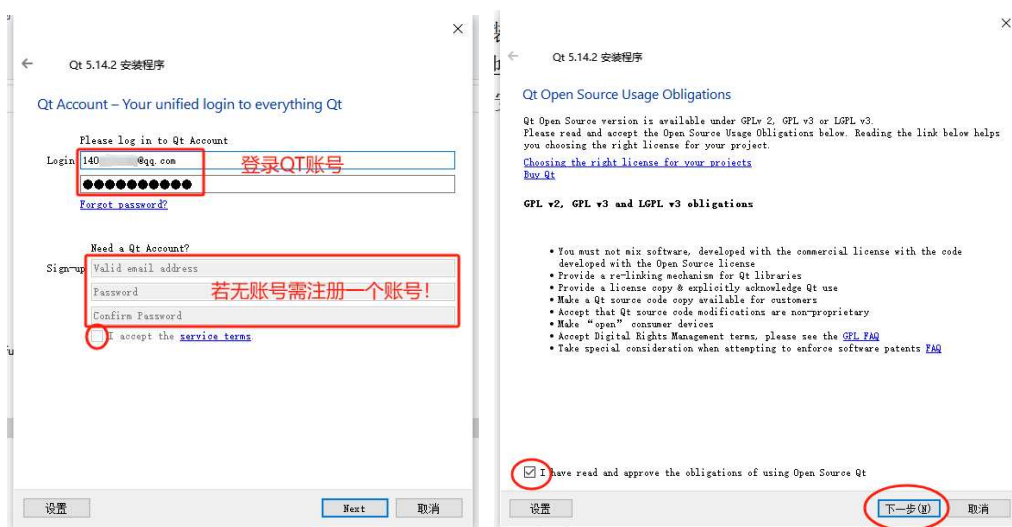
typedef enum {
    STATUS_SUCCESS = 0,
    STATUS_ERROR_PARAM,
    STATUS_ERROR_TIMEOUT,
    STATUS_ERROR_MEMORY,
} StatusCode; // 定义错误码 (status.h)
StatusCode ReadSensorData(float* data);

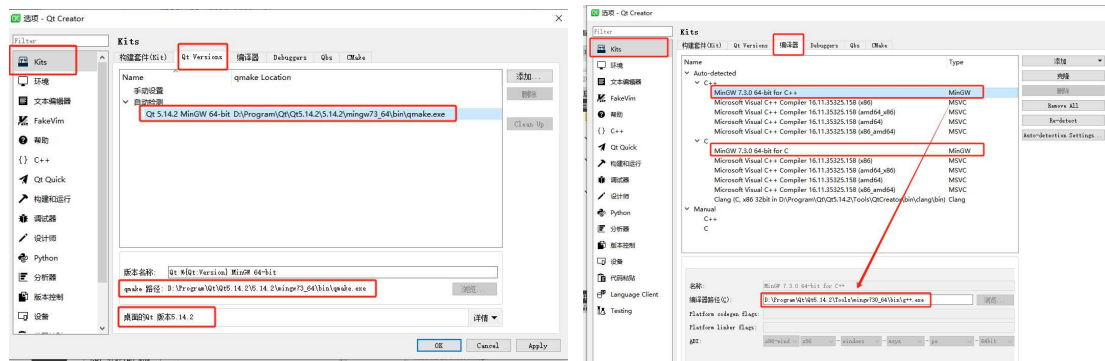
```

二、第一个 QT helloworld 工程

1.QT 5.14 下载&安装

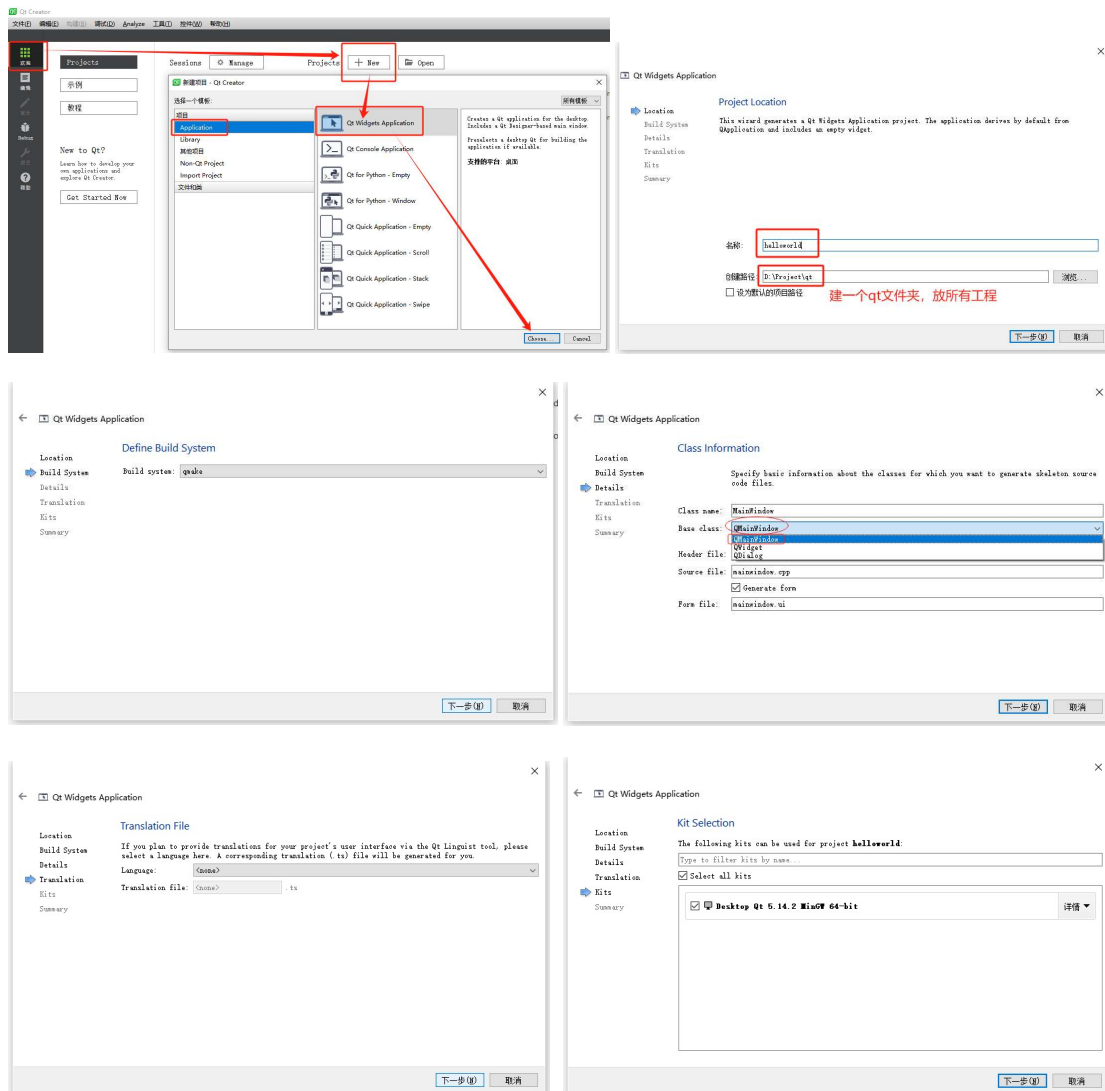
QT 官网地址：<https://download.qt.io/>，中国大陆的下载被限制，选择实验室或第三方提供的安装包安装 QT。QtCreator 5.14.2 版本是最后一个可以离线安装的版本，之后的版本只能通过在线安装模式进行安装。QT-5.14.2 网盘百度链接（提取码：0206）：<https://pan.baidu.com/s/1-Ah9fMnK2C1u2cRxXI7Uiw?pwd=0206>

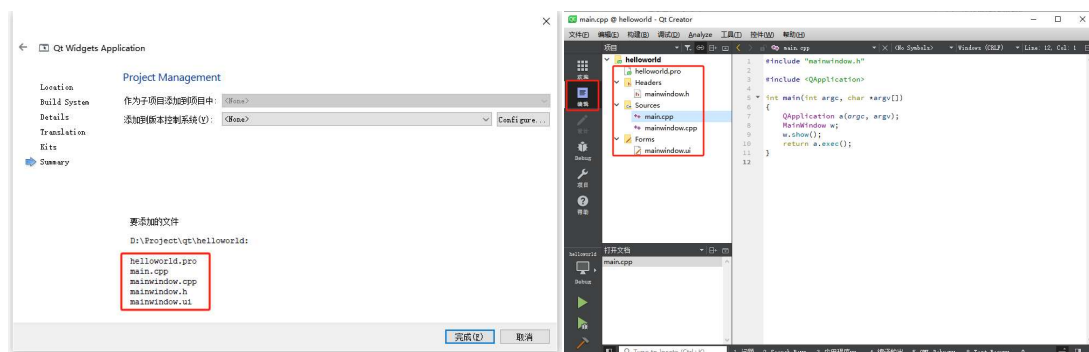




3.新建一个 helloworld Qt 工程

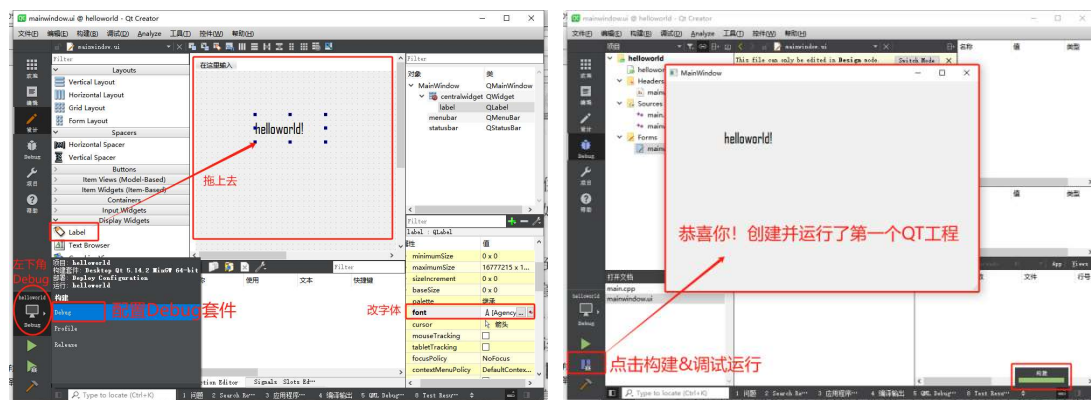
工程目录创建在“D:/project/qt”路径下，该路径中将存放工程的源文件。



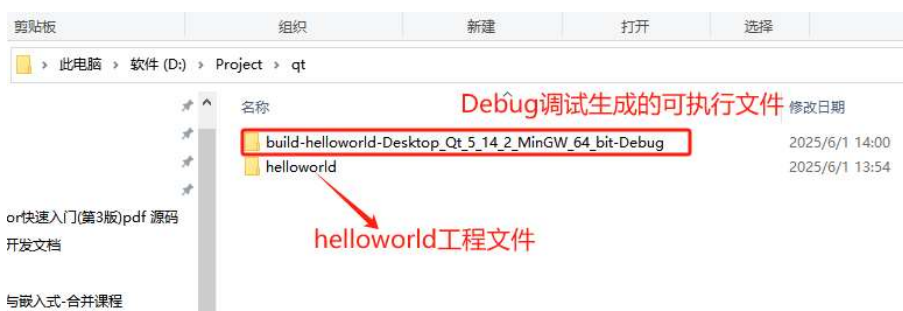


4.编辑、构建（编译）、调试 helloworld 工程

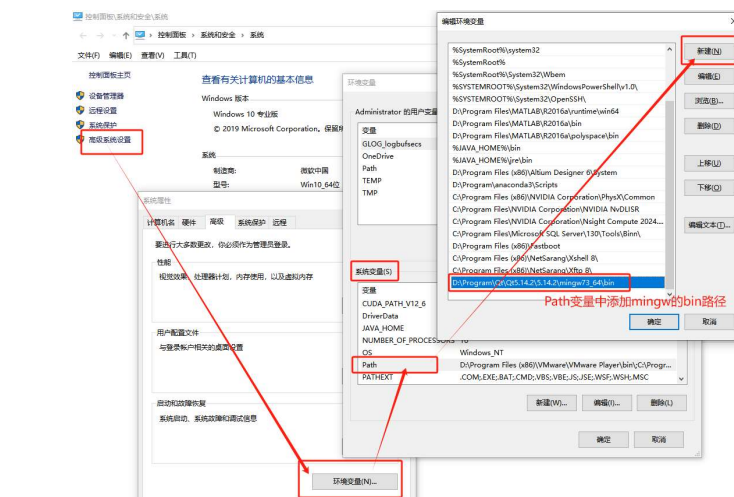
工程创建完成后，可先在 Forms 下 mainwindow.ui 文件中添加一个 Label 标签显示“helloworld!”字符样式，再在左下 Debug 栏中配置 Debug 套件，说明后续将采用 Qt Desktop 级开发工具集（Desktop Qt MinGW 64-bits）来构建（编译）、调试工程。



点击左下角的锤子（构建）将编译工程，点击其上方的绿色三角形（构建并调试运行），将弹出一个显示“helloworld!”样式的主窗口。另外，在“D:/project/qt”路径下将生成一个以“***-Debug”命名的文件夹，其中包含调试生成的可执行文件。

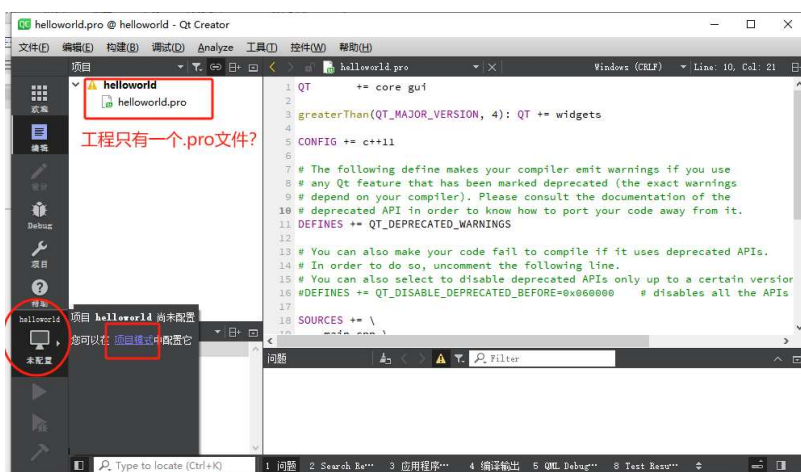


双击 helloworld.exe 可执行文件，windows 系统中也将弹出一个“helloworld!”字符样式的主窗口。若提示缺少 Qt5Cored.dll 文件，直接将 Qt 的 bin 目录路径加入到系统 Path 环境变量中解决。



5.问题：工程创建后只有一个.pro 文件

若工程创建后只有一个.pro 文件，说明 QtCreator 安装过程中勾选的组件有误，需要重新安装。



三、QT 基础知识

请参考《Qt Creator 快速入门》（第3版）之“第3章 窗口布局”、“第4章 布局管理”、“第5章 应用程序主窗口”、“第6章 事件系统”、“第7章 Qt对象模型与容器类”、“第8章 界面外观”等内容。或者，也可查阅《华清远见 S5P6818 Qt 开发教程》之“第2章 Qt 开发控件介绍”、“第3章 Qt 开发功能介绍”等内容，先快速过一遍基础知识，后续用到再细看相关章节，重点掌握“3.1 信号与槽”、“3.2 秒表”、“3.3 文件操作”、“3.4 数据库”、“3.5 客户端-服务器 TCP 通信”、“3.6 UDP 广播消息”等内容。

Qt 系列基础教程：

https://www.bilibili.com/video/BV1N34y1H7x7?spm_id_from=333.788.videopod.episodes&vd_source=851bcabe2785dea719d0bc99c51e5038

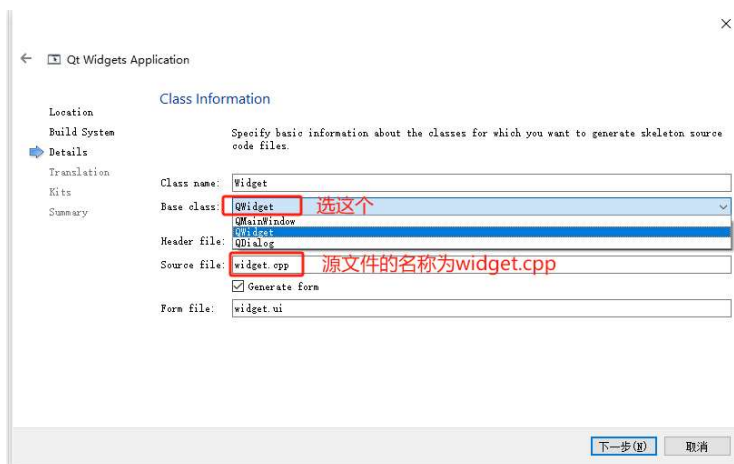
四、QT 串口助手开发(参考 B 站视频)

QT 串口助手开发:

https://www.bilibili.com/video/BV1NfTvzLETK/?share_source=copy_web&vd_source=31ae21efa46e218d8c39b918ca5d3c17

1.创建 QT 工程

BaseClass 栏选择 QWidget。



2.设计 UI（照葫芦画瓢）



主窗口大小（Widget）：550*400

接收窗口（Text Browser）：350*200

发送窗口（Text Edit）：350*70

按钮（Push Button）：刷新、状态指示灯、打开串口、4 种格式、发送、2 个清空

标签（Label）：波特率标签、提示信息 message、TX、RX

下拉列表（Combo Box）：COM 口、波特率值

标题（windowTitle）：My 串口助手

背景样式更改：

所有按键变灰色凸起 styleSheet: `background-color:rgb(232,232,232);`

串口状态按键（LED） styleSheet 变绿色: `"background-color: green;"`

右下标签（message）字体 styleSheet 变红色 `"color: red;"`

字符串或 Hex 格式按键 styleSheet 变绿色: `"background-color: green;"`

3.编写后端代码

（1）.pro 文件中添加 serialport 串口模块。

（2）在 widget.h 中 Widget 类中添加一个 QSerialPort 串口连接对象，用于后续串口连接操作。QSerialPortInfo 类为串口查询类，可直接调用其静态函数，不用添加私有成员对象。

（3）在 UI 中添加槽函数：

- 刷新按键：先清空 comboBox 串口列表，查询新的串口名字列表，将列表添加到 comboBox 中显示。
- 打开串口按键：先设置串口工作参数，再打开串口，最后判断串口是否已成功打开。若已成功打开，将串口状态按键 LED 变绿色，标签 message 改为“串口已成功打开！”，打开串口按键改为“关闭串口”。

- 串口接收到字符（手动添加槽函数，没有对应的按键）：读取串口中的所有二进制字符串，按字符串或 HEX 格式在文本浏览框末尾显示，最后将光标移动到文本最后。

其中，`QString::arg()`是字符串格式化函数，`QString("姓名: %1, 年龄: %2").arg("张三").arg(25)`表示将"张三"和 25 分别插入到字符串中不同位置。

- 发送按键：先判断串口是否已打开，再按字符串或 HEX 格式将文本编辑框中的字符发送出去，最后统计发送字符的总数。

其中，在打开串口按键的槽函数中添加一行，为串口接收字符的槽函数设置一个触发信号。

//信号触发者为串口，发出信号的触发条件是当有新数据可读时，槽函数的接收者为窗口自身，响应信号的槽函数地址

```
connect(&m_serial, &QSerialPort::readyRead, this, &Widget::messageReceived);
```

- 4 个格式按键：HEX 显示、字符串显示、HEX 发送、字符串发送。设置相应的 styleSheet，按下的那个变绿，另一个变灰。设置接收和发送的状态。